[Nombre de la universidad]

Facultad o programa: [Nombre del programa]

Arquitectura de Software

Unidad 2 · Mini proyecto (Aprendizaje Basado en Proyectos)

Tejiendo redes: arquitectura de software entre hilos y nodos

Facturación electrónica distribuida para el consultorio médico «Salud Integral»

Informe técnico

Integrante	Rol asignado
José Ignacio Gómez Guzmán	[Rol]
[Nombre del integrante 2]	[Rol]
[Nombre del integrante 3]	[Rol]
[Nombre del integrante 4]	[Rol]
[Nombre del integrante 5 (si aplica)]	[Rol]

Docente: [Nombre del docente]

Fecha de entrega: 11 de noviembre de 2026

Repositorio: https://github.com/JNACHO2001/consultorio-medico
Video: [enlace de YouTube no listado]

Índice

1. Introducción....4
1.1 Contexto del proyecto....4
1.2 Propósito del diseño de la arquitectura....4
1.3 Objetivos....4
1.4 Alcance y límites....5
2. Descripción del sistema distribuido....5
2.1 Tipo de sistema....5
2.2 Componentes funcionales....5
2.3 Supuestos....6
2.4 Herramientas seleccionadas....6
3. Análisis y planificación del proyecto....6
3.1 Definición de requisitos....6
3.2 Relevancia de la comunicación entre nodos e hilos....7
3.3 Metas del sistema....8
3.4 Estrategia del equipo y roles....8
3.5 Cronograma....8
3.6 Riesgos....9
4. Modelado UML....9
4.1 Diagrama de componentes....9
4.2 Diagrama de secuencia....10
4.3 Diagrama de despliegue....12
5. Selección y justificación de patrones de diseño....12
5.1 Proxy....13
5.2 Mediator....13
5.3 Observer....14
5.4 Circuit Breaker y Retry....14
5.5 Idempotent Receiver....14
5.6 Competing Consumers, pool de hilos y contrapresión....14
5.7 Patrones heredados de la Unidad 1....14
6. Implementación y simulación del prototipo distribuido....15
6.1 Estructura del código....15
6.2 Protocolo de comunicación....15
6.3 Hilos y procesos....15
6.4 Cómo se simula el entorno distribuido....16
6.5 Integración con el sistema existente....16
6.6 Cómo ejecutarlo....17
6.7 Pruebas automáticas....17
7. Evaluación del desempeño y ajustes....17
7.1 Metodología....17
7.2 Escalabilidad horizontal (e1)....18
7.3 Escalabilidad vertical: hilos por nodo (e2)....19
7.4 Saturación y contrapresión (e3)....20
7.5 Robustez ante la caída de un nodo (e4)....21
7.6 Servicio completo y el ajuste del candado (e5)....22

7.7 Cuellos de botella y ajustes....23
7.8 Resultado de las pruebas automáticas....24
7.9 Límites y amenazas a la validez....24
8. Conclusiones....24
8.1 Reflexión sobre las decisiones y los desafíos....25
8.2 Trabajo futuro....25
Referencias....26
Anexo A. Trazabilidad con los criterios de evaluación....27
Anexo B. Enlaces de la entrega....27

1. Introducción

1.1 Contexto del proyecto

El consultorio médico «Salud Integral» es el proyecto que el equipo desarrolló en la Unidad 1: un sistema web que centraliza pacientes, citas, historias clínicas, ingresos, facturación, convenios con EPS, cartera, órdenes de apoyo diagnóstico, caja y facturación electrónica. Está construido en Python con FastAPI, organizado por capas y con el patrón Repository como eje, y su prototipo está publicado en un repositorio de GitHub y desplegado en la nube.

En esa primera versión todo se ejecuta en un solo proceso. Para la Unidad 2 («Tejiendo redes: arquitectura de software entre hilos y nodos») el reto es llevar una parte real del sistema a una arquitectura distribuida, en la que varios componentes repartidos en una red cooperan para cumplir un objetivo común, y evaluarla con rigor.

La parte elegida es la facturación electrónica. Es el único servicio del consultorio que depende de un tercero remoto: un proveedor tecnológico que valida, firma y transmite cada factura o nota crédito a la DIAN. Ese proveedor es lento (decenas o cientos de milisegundos por documento), puede fallar de forma independiente y es crítico, porque por la regla de negocio RN-22 no se puede radicar una factura al pagador sin factura electrónica aceptada. Reúne, por eso, los problemas clásicos de la computación distribuida: latencia de red, fallos parciales, concurrencia, balanceo de carga y necesidad de reintentos seguros.

1.2 Propósito del diseño de la arquitectura

El propósito es diseñar, modelar, implementar y evaluar una arquitectura en la que el envío de documentos electrónicos se reparta entre nodos proveedor independientes, cada uno con su propio pool de hilos, sin que el resto del sistema note la diferencia, y demostrar con mediciones que esa arquitectura escala, tolera la caída de un nodo y degrada con elegancia cuando falla.

1.3 Objetivos

Objetivo general. Diseñar y evaluar una arquitectura de software distribuida para la facturación electrónica del consultorio «Salud Integral», aplicando modelado UML y patrones de diseño, e implementar un prototipo que simule la interacción de nodos e hilos en una red.

Objetivos específicos.

- Modelar la arquitectura con UML: diagrama de componentes, de secuencia (nodos e hilos) y de despliegue.
- Seleccionar y justificar patrones de diseño pertinentes (Proxy, Mediator, Observer, Circuit Breaker, Retry e Idempotent Receiver) con base en la bibliografía.
- Implementar un prototipo con nodos como procesos independientes, pool de hilos por nodo y comunicación por TCP, integrado al sistema existente sin romper sus pruebas.
- Evaluar escalabilidad horizontal y vertical, saturación y robustez ante la caída de un nodo, documentando métricas y cuellos de botella.
- Aplicar los ajustes que la evaluación justifique y medir su efecto.

1.4 Alcance y límites

El proveedor tecnológico de los nodos es simulado: valida localmente las reglas básicas del documento, espera un tiempo configurable (por defecto 40 ms) para representar la firma y la transmisión, y no habla con la DIAN real. Los nodos de las mediciones corren como procesos del sistema operativo en una misma máquina de 2 núcleos; la red es el bucle local (loopback). El informe señala en la sección 7 qué conclusiones sobreviven a esa simplificación y cuáles no.

2. Descripción del sistema distribuido

2.1 Tipo de sistema

Es un servicio distribuido de procesamiento con un grupo de trabajadores remotos: la API recibe la solicitud de emisión, y un proxy reparte el trabajo entre N nodos que lo ejecutan en paralelo. La comunicación es síncrona (petición y respuesta) sobre TCP con un mensaje JSON por línea. Cada nodo atiende varias solicitudes a la vez con un pool de hilos, de modo que el sistema combina dos niveles de concurrencia: entre nodos (procesos distintos, escalado horizontal) y dentro de cada nodo (hilos, escalado vertical).

2.2 Componentes funcionales

Tabla 1
Componentes del sistema distribuido

Componente	Responsabilidad	Dónde corre
`FacturacionElectronicaService`	Numera el documento, calcula CUFE y XML, lo guarda como GENERADO y llama al proveedor; guarda la respuesta.	API (proceso uvicorn)
`ProveedorProxy`	Representa al proveedor remoto: serializa, envía, reintenta con espera exponencial y jitter, y degrada si no hay nodos.	API
`MediadorNodos`	Registro de nodos, elección del menos cargado, circuit breaker por nodo y latidos.	API
`ClienteNodo`	Pool de conexiones TCP persistentes hacia un nodo.	API
`BusEventos` y `Metricas`	Publicación asíncrona de eventos y cálculo de totales, reparto por nodo y percentiles.	API (hilo propio)
`NodoProveedor`	Servidor TCP con pool de hilos, cola acotada, caché de idempotencia y proveedor simulado.	N procesos independientes
`ClusterLocal`	Lanza N nodos como procesos del sistema operativo (demostración, pruebas y mediciones).	Máquina del desarrollador

2.3 Supuestos

- **Modelo de fallos.** Los nodos fallan por parada (el proceso muere o deja de responder); no se consideran nodos que mientan o corrompan datos.
- **Red.** Puede rechazar conexiones, cortarlas o tardar; no se supone sincronía de relojes entre la API y los nodos.
- **Idempotencia.** Enviar dos veces el mismo documento al proveedor no debe producir dos documentos; el nodo lo garantiza con una clave de idempotencia.
- **Una sola API.** Los datos del consultorio siguen en memoria dentro de un proceso (Unidad 1). Los nodos no guardan datos clínicos ni contables: solo reciben el documento a validar.
- **Latencia representativa.** 40 ms por documento representa la firma y la transmisión al proveedor; el valor es configurable.
- **Confianza en la red interna.** En el prototipo no hay cifrado ni autenticación entre la API y los nodos; en producción se exigirían TLS mutuo o una red privada.

2.4 Herramientas seleccionadas

Tabla 2
Herramientas y justificación

Herramienta	Uso	Motivo
Python 3.13 (biblioteca estándar)	`socketserver`, `concurrent.futures`, `threading`, `socket`	Hilos, pool y TCP reales sin dependencias nuevas; el proyecto ya es Python.
FastAPI y uvicorn	API REST existente	Se conserva; solo se añade configuración, un endpoint de estado y el arranque del clúster.
pytest	181 pruebas automáticas	Verifican el contrato y las reglas, y ahora el comportamiento distribuido.
Procesos del sistema operativo	Nodos reales (`subprocess`)	Permiten matar un nodo con SIGKILL y medir la recuperación.
Matplotlib y Graphviz	Diagramas UML a color y gráficas	Reproducibles desde un script del repositorio.
GitHub, Docker y Render	Repositorio y despliegue	Ya usados en la Unidad 1; el modo local sigue desplegándose igual.

3. Análisis y planificación del proyecto

3.1 Definición de requisitos

Los requisitos de la Unidad 2 se agregan a los de la Unidad 1 (RF-01 a RF-27 y RN-01 a RN-26). Se identifican con el sufijo D de «distribuido».

Tabla 3
Requisitos del sistema distribuido

Código	Requisito	Verificación
RFD-01	La factura electrónica se emite por medio de nodos remotos sin cambiar el servicio de facturación.	Prueba de integración por la API
RFD-02	El número de nodos es configurable (`NODOS_PROVEEDOR`) y la carga se reparte entre ellos.	Prueba de balanceo con 3 nodos
RFD-03	Si un nodo falla, se agota el tiempo o está saturado, el envío se reintenta en otro nodo.	Pruebas de caída, lentitud y saturación
RFD-04	Reintentar no duplica el trabajo (idempotencia).	Pruebas de clave repetida y simultánea
RFD-05	Sin nodos disponibles, el documento queda RECHAZADO (NODOS-01) y se reenvía con «Reintentar».	Prueba de API sin nodos y recuperación
RFD-06	El administrador puede ver el estado de los nodos y las métricas; `/health` informa los nodos disponibles.	Prueba del endpoint de nodos
RNFD-01	Escalabilidad horizontal casi lineal hasta 4 nodos.	Experimento e1
RNFD-02	Tolerancia a la caída de un nodo sin perder documentos.	Experimento e4
RNFD-03	Contrapresión: una cola de admisión acotada, no una cola infinita.	Experimento e3 y pruebas del nodo
RNFD-04	La numeración de facturas es única aunque se emita en paralelo.	Prueba concurrente y experimento e5
RNFD-05	Sin `NODOS_PROVEEDOR` el sistema funciona como en la Unidad 1 (Render).	Las 155 pruebas previas siguen pasando

3.2 Relevancia de la comunicación entre nodos e hilos

La emisión de un documento es un trabajo limitado por espera (red y proveedor), no por cálculo. Por eso los hilos son la herramienta correcta: mientras un hilo espera la respuesta del proveedor, otros avanzan. En Python el bloqueo global del intérprete (GIL) impide que varios hilos ejecuten bytecode a la vez, pero se libera durante las esperas de entrada y salida, de modo que un pool de hilos sí multiplica el rendimiento de un trabajo que mayormente espera.

La capacidad de un nodo se puede predecir con la ley de Little (Little, 1961): si cada documento ocupa un hilo durante W segundos y hay h hilos, el nodo procesa como máximo h / W documentos por segundo. Con W = 40 ms, un nodo de 4 hilos admite unos 100 documentos por segundo, y N nodos, unos 100·N. La sección 7 contrasta esta predicción con las mediciones.

Los nodos (procesos separados) añaden lo que los hilos no dan: aislamiento de fallos (si un nodo muere, los demás siguen), escalado más allá de una máquina y despliegue independiente. A cambio aparecen los problemas de la distribución: la comunicación puede fallar a medias, las respuestas pueden perderse y los reintentos pueden duplicar trabajo; de ahí que el diseño incluya idempotencia, circuit breaker y degradación controlada (Tanenbaum y van Steen, 2017).

3.3 Metas del sistema

- Eficiencia de escalado horizontal de al menos 90 % al pasar de 1 a 4 nodos.
- Tasa de éxito del 100 % cuando cae 1 de 3 nodos en plena carga, con recuperación automática del nodo sin reiniciar la API.
- Capacidad de agregar nodos solo cambiando una variable de entorno.
- Ningún consecutivo repetido ni documento perdido bajo concurrencia.
- Cero cambios en el comportamiento observable del sistema cuando no hay nodos configurados.

3.4 Estrategia del equipo y roles

El equipo (4 a 5 integrantes) trabajó con roles definidos y entregables por rol. Los nombres de quienes ocupan cada rol figuran en la portada.

Tabla 4
Roles del equipo

Rol	Responsabilidades	Entregables principales
Líder de proyecto	Plan, seguimiento del cronograma, integración de entregables y entrega en el LMS.	Plan de proyecto, entrega final
Diseñador de arquitectura	Diagramas UML, selección y justificación de patrones.	Secciones 4 y 5, diagramas
Desarrollador	Implementación de nodos, proxy, mediador y observadores; integración con la API.	Paquete `app/distribuido`, repositorio
Evaluador de calidad	Pruebas automáticas, experimentos de carga y análisis de cuellos de botella.	Pruebas, `benchmarks/`, sección 7
Documentación y video	Informe, referencias APA 7 y presentación en video.	Este informe, video no listado

La estrategia fue iterativa y guiada por mediciones: se construyó primero el camino completo más simple (un nodo, un envío), se cubrió con pruebas, se añadió tolerancia a fallos y, solo después, se midió. Los ajustes se decidieron con los números y no por intuición.

3.5 Cronograma

Tabla 5
Cronograma tentativo

Fase	Actividades	Duración	Resultado
1. Análisis y plan	Elegir el sistema, definir requisitos RFD y RNFD, roles y herramientas.	Semana 1	Plan y requisitos
2. Diseño	Diagramas de componentes, secuencia y despliegue; selección de patrones.	Semana 1	Sección 4 y 5
3. Implementación	Nodo, protocolo, proxy, mediador, observadores, integración.	Semana 2	Prototipo funcional

Fase	Actividades	Duración	Resultado
4. Evaluación y ajustes	Pruebas automáticas, experimentos e1 a e5, ajuste del candado y de la cola.	Semana 2	Sección 7
5. Informe y video	Informe técnico, guion y grabación del video de 12 a 15 minutos.	Semana 3	PDF/Word y video
6. Revisión y entrega	Coherencia de artefactos y carga en el LMS.	Semana 3	Entrega

3.6 Riesgos

Tabla 6
Riesgos identificados

Riesgo	Efecto	Mitigación
Reintentos que duplican trabajo	Documentos repetidos o consecutivos duplicados	Clave de idempotencia en el nodo y reserva del consecutivo con candado
Un nodo lento o caído	Latencia alta o pérdida de documentos	Tiempo de espera, circuit breaker y reintento en otro nodo
Sobrecarga de un nodo	Colas ilimitadas y memoria creciente	Cola de admisión acotada con respuesta OCUPADO
Mediciones poco representativas	Conclusiones engañosas	Repeticiones, calentamiento y límites declarados en la sección 7

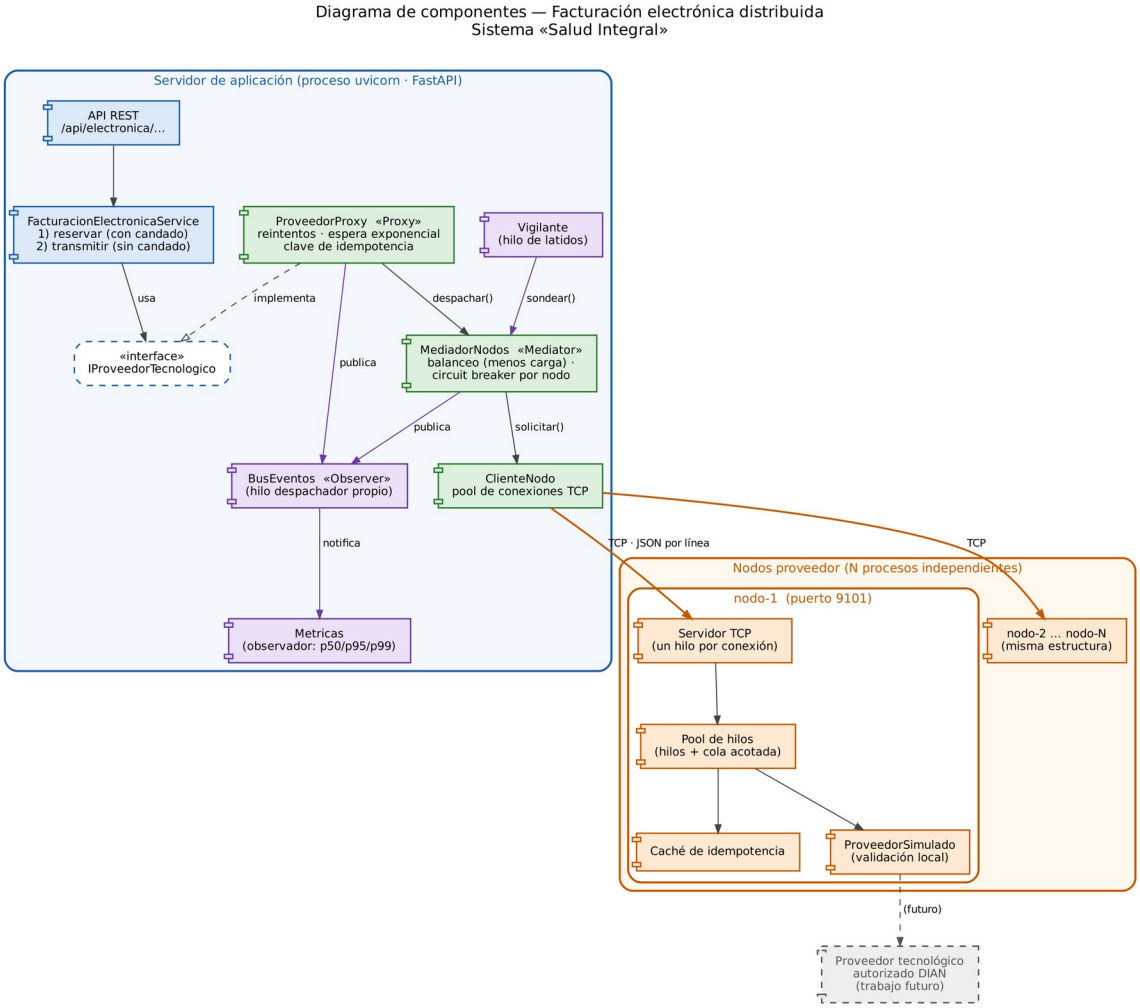
4. Modelado UML

Los diagramas se generan con el script `docs/uml/render_distribuido.py` (Graphviz y Matplotlib) y se presentan a color. Complementan los diagramas de la Unidad 1 (casos de uso, clases, secuencia de agendar una cita y despliegue), que siguen vigentes.

4.1 Diagrama de componentes

El diagrama de componentes separa dos grandes bloques: el servidor de aplicación y los nodos proveedor. Dentro del servidor, `FacturacionElectronicaService` solo conoce la interfaz `IProveedorTecnologico`; el `ProveedorProxy` la implementa y delega en el `MediadorNodos`, que usa un `ClienteNodo` por nodo. Proxy y mediador publican hechos en el `BusEventos`, que los entrega a `Metricas`, y un `Vigilante` envía latidos. Cada nodo es un proceso con servidor TCP, pool de hilos, caché de idempotencia y proveedor simulado.

Figura 1
Diagrama de componentes de la facturación electrónica distribuida



Nota. Elaboración propia. Verde: patrones Proxy y Mediator; morado: Observer; naranja: nodos; azul: servicio y API.

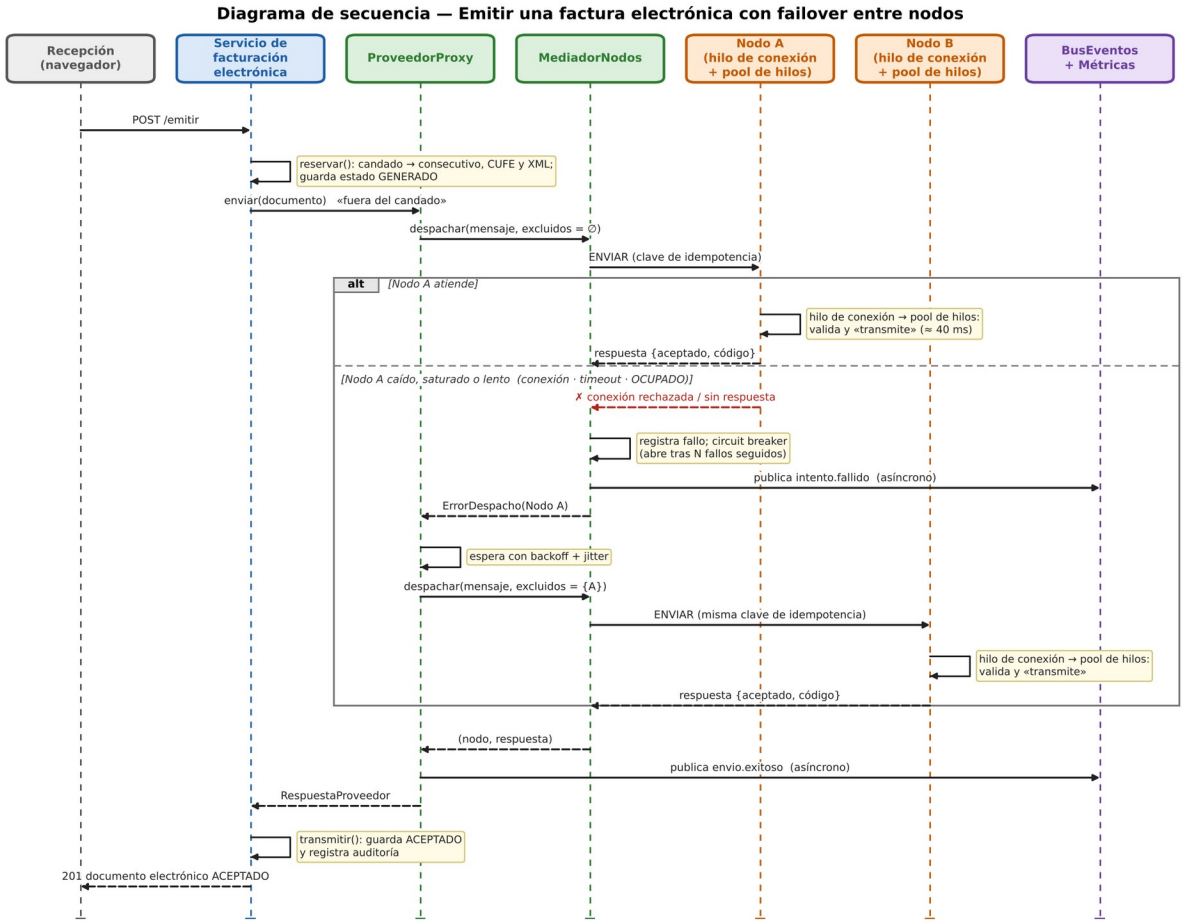
Explicación técnica. La dependencia clave es la inversión: el servicio depende de una abstracción (`IProveedorTecnologico`) y no de dónde corre el proveedor. Cambiar de proveedor local a distribuido es una decisión de composición en `container.py`, igual que en la Unidad 1 se cambiaba de repositorio en memoria a SQL. Las flechas hacia los nodos son el único punto donde hay red; todo lo demás es llamada local.

4.2 Diagrama de secuencia

El diagrama de secuencia sigue una emisión completa, incluyendo el caso en que el primer nodo falla. Muestra también qué ocurre dentro de cada nodo: un hilo de conexión recibe la solicitud y la entrega al pool de hilos, que hace el trabajo.

Figura 2

Diagrama de secuencia: emisión de una factura electrónica con failover entre nodos



Nota. Elaboración propia. El marco *alt* distingue el nodo A que atiende del que falla; las flechas de publicación al bus son asíncronas.

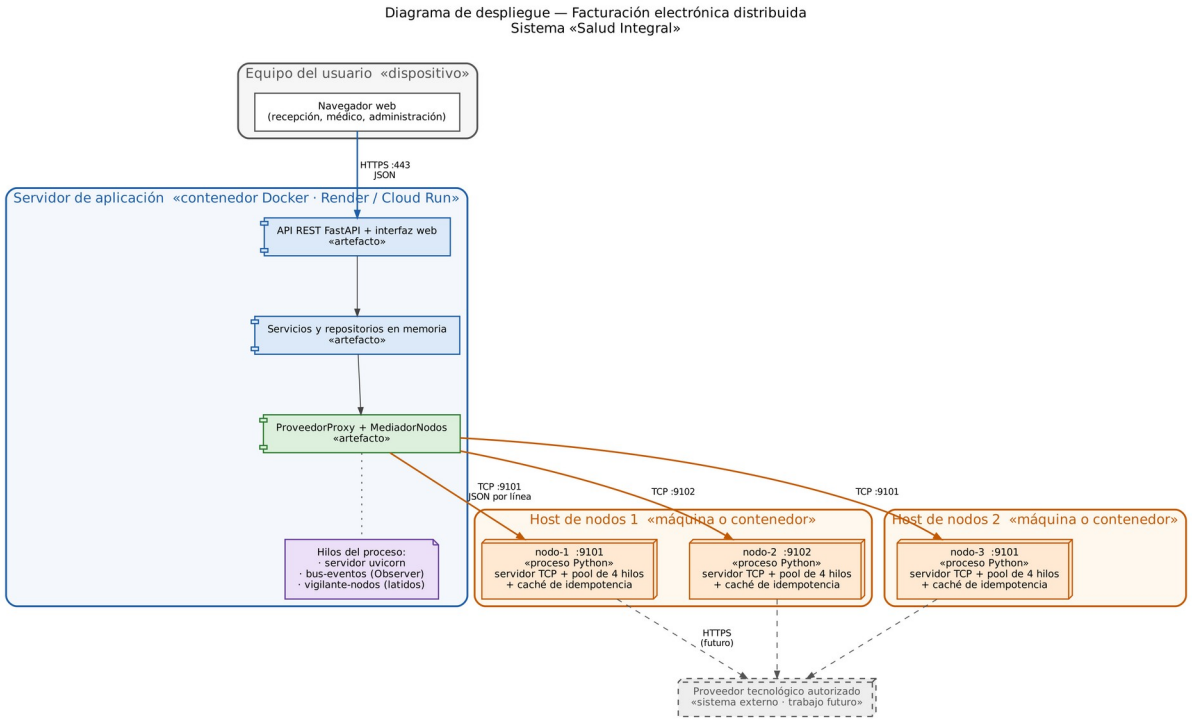
Explicación técnica.

- **Reserva con candado.** El servicio numera el documento (consecutivo, CUFE, XML) y lo guarda como GENERADO dentro de un candado corto. Así el consecutivo queda reservado y cualquier otra emisión de la misma factura recibe un conflicto.
- **Envío fuera del candado.** La transmisión es lenta; si el candado la cubriera, se serializarían todas las emisiones (cuello de botella medido en la sección 7).
- **Despacho.** El mediador elige el nodo con menos solicitudes en curso. El mensaje lleva una clave de idempotencia.
- **Fallo y reintento.** Ante conexión rechazada, tiempo agotado u OCUPADO, el mediador registra el fallo (y abre el circuito tras varios seguidos) y el proxy espera con backoff y jitter antes de probar otro nodo con la misma clave.
- **Observación.** Los eventos se publican sin esperar a los observadores, de modo que medir no frena la emisión.

4.3 Diagrama de despliegue

El despliegue muestra la distribución física: el navegador del usuario, el servidor de aplicación en un contenedor y los nodos proveedor, que pueden compartir host o repartirse en varios. La API abre conexiones TCP hacia cada nodo; los nodos, en el futuro, hablarían con el proveedor tecnológico real.

Figura 3
Diagrama de despliegue de la facturación electrónica distribuida



Nota. Elaboración propia. Línea punteada: sistema externo fuera del alcance del prototipo.

Explicación técnica. La lista de nodos es la única información de topología y vive en la variable `NODOS_PROVEEDOR` (por ejemplo, `host1:9101,host1:9102,host2:9101`). Agregar capacidad es levantar otro nodo y añadir su dirección. El despliegue en Render de la Unidad 1 no cambia porque, sin esa variable, el sistema usa el proveedor simulado dentro del mismo proceso.

5. Selección y justificación de patrones de diseño

Los patrones se eligieron por el problema concreto que resuelven en este sistema, no por catálogo. La tabla resume la selección y las subsecciones la justifican con la bibliografía.

Tabla 7
Patrones aplicados

Patrón	Dónde	Problema que resuelve	Fuente
Proxy (remoto)	`ProveedorProxy`	Ocultar que el proveedor está en otra máquina	Gamma et al., 1994

Patrón	Dónde	Problema que resuelve	Fuente
Mediator	`MediadorNodos`	Que los clientes no conozcan a cada nodo ni su estado	Gamma et al., 1994
Observer	`BusEventos`, `Metricas`	Observar sin acoplar ni frenar la emisión	Gamma et al., 1994
Circuit Breaker	`MediadorNodos`	No insistir contra un nodo caído	Nygard, 2018; Microsoft, s. f.
Retry con backoff y jitter	`ProveedorProxy`	Superar fallos transitorios sin avalancha de reintentos	Microsoft, s. f.
Idempotent Receiver	`NodoProveedor`	Reintentar sin duplicar trabajo	Hohpe y Woolf, 2003
Competing Consumers y contrapresión	Pool de hilos y cola acotada	Procesar en paralelo sin colas infinitas	Hohpe y Woolf, 2003; Tanenbaum y van Steen, 2017
Adapter (heredado)	`IProveedorTecnologico`	Cambiar de proveedor sin tocar el servicio	Gamma et al., 1994
Repository (heredado)	Repositorios en memoria	Independencia del almacenamiento	Fowler, 2002; Evans, 2003
Health Endpoint Monitoring (extendido)	`/health`, `/api/electronica/nodos`	Que la plataforma detecte fallas	Microsoft, s. f.

5.1 Proxy

Un proxy remoto es un representante local de un objeto que está en otro espacio de direcciones (Gamma et al., 1994). `ProveedorProxy` cumple `IProveedorTecnologico`, de modo que el servicio lo usa como si fuera el proveedor simulado local. Es además la pieza donde viven las políticas de comunicación: serialización, tiempo de espera, reintentos y la degradación. Consecuencia: una llamada que parece local puede tardar o fallar; por eso el proxy nunca lanza una excepción de red hacia arriba, sino que devuelve un rechazo NODOS-01 que el sistema ya sabe tratar con «Reintentar».

5.2 Mediator

El mediador encapsula cómo interactúa un conjunto de objetos y evita que se refieran entre sí de forma explícita (Gamma et al., 1994). Aquí el proxy solo dice «despacha este mensaje»; el `MediadorNodos` conoce los nodos, elige el que tiene menos solicitudes en curso (con reparto por turnos en los empates) y mantiene su estado de salud. Consecuencia: la política de balanceo y los circuitos se cambian en un único lugar; el riesgo, un mediador demasiado grande, se contiene porque no hace red por sí mismo (delega en `ClienteNodo`).

5.3 Observer

Observer define una dependencia uno a muchos para que, al cambiar un objeto, sus dependientes se enteren sin acoplarse (Gamma et al., 1994). `BusEventos` publica hechos del sistema distribuido (envío exitoso, intento fallido, cambio de circuito) y entrega cada evento a sus suscriptores, entre ellos `Metricas`, desde un hilo propio con cola acotada. Si la cola se llena, el evento se descarta y se cuenta: la observabilidad nunca debe tumbar la operación. Limitación declarada: el bus vive dentro del proceso de la API; es un Observer sobre eventos originados en un sistema distribuido. Para observar entre procesos habría que reemplazarlo por un intermediario de mensajes, algo que la interfaz de suscripción permite sin cambiar a los observadores.

5.4 Circuit Breaker y Retry

El circuit breaker evita que un cliente siga llamando a un servicio que probablemente falle, y le da tiempo de recuperarse (Nygard, 2018). Cada nodo tiene tres estados: CERRADO (recibe tráfico), ABIERTO (se descarta durante un enfriamiento) y SEMIABIERTO (deja pasar una sola solicitud de prueba). Se abre tras `CIRCUITO_FALLOS` fallos seguidos (3 por defecto) y un nodo saturado no cuenta como caído: responde, solo que sin cupo. El reintento usa espera exponencial con jitter para que muchos clientes no reintenten a la vez (Microsoft, s. f.): base de 50 ms, tope de 1 s y factor aleatorio entre 0,5 y 1,5.

5.5 Idempotent Receiver

Si un cliente no recibe respuesta, no sabe si el servidor ejecutó o no la operación; reintentar sin más puede duplicarla. Un receptor idempotente reconoce los mensajes repetidos y no repite el efecto (Hohpe y Woolf, 2003). El nodo guarda por clave (`tipo:número:CUFE`) el resultado ya calculado y, si una clave llega mientras otra igual está en proceso, comparte el resultado en lugar de procesarla dos veces. Las pruebas cubren ambos casos.

5.6 Competing Consumers, pool de hilos y contrapresión

Un pool de hilos fijo es la forma estándar de que varios consumidores compitan por el trabajo sin crear un hilo por solicitud (Hohpe y Woolf, 2003; Python Software Foundation, s. f.-a). La decisión de diseño importante es que la cola de admisión está acotada: caben `hilos + cola` solicitudes; la siguiente recibe OCUPADO de inmediato. Es contrapresión explícita: es preferible que el nodo diga «no puedo» y el proxy derive el trabajo a otro nodo, a que acepte sin límite y degrade para todos. La sección 7 muestra el costo de acotar demasiado la cola y el efecto de ajustarla.

5.7 Patrones heredados de la Unidad 1

El Adapter (`IProveedorTecnologico`) fue lo que permitió insertar el proxy sin tocar el servicio. El Repository no cambia: los documentos siguen en el repositorio en memoria y la reserva del consecutivo se apoya en sus operaciones atómicas. El Health Endpoint Monitoring se extiende: `/health` informa cuántos nodos están disponibles sin alterar su código de estado (la API sigue viva aunque los nodos estén caídos) y un endpoint de administración detalla el circuito y las métricas de cada nodo.

6. Implementación y simulación del prototipo distribuido

6.1 Estructura del código

Tabla 8
Archivos del módulo distribuido

Archivo	Contenido
`backend/app/distribuido/protocolo.py`	Formato de mensajes, serialización del documento y clave de idempotencia.
`backend/app/distribuido/nodo.py`	`NodoProveedor`: servidor TCP, pool de hilos, cola acotada, caché; ejecutable como proceso.
`backend/app/distribuido/mediador.py`	`MediadorNodos`, `ClienteNodo` y circuit breaker.
`backend/app/distribuido/proxy.py`	`ProveedorProxy`: reintentos, backoff y degradación.
`backend/app/distribuido/eventos.py`	`BusEventos` y `Metricas` (Observer).
`backend/app/distribuido/cluster.py`	`ClusterProveedor`, `ClusterLocal` y `NodoProceso` (lanzador de procesos).
`backend/tests/test_distribuido.py`	26 pruebas del módulo.
`backend/benchmarks/`	Experimentos, resultados en JSON y gráficas.
`docs/uml/render_distribuido.py`	Generador de los diagramas UML a color.

6.2 Protocolo de comunicación

Cada mensaje es un objeto JSON en una línea terminada en salto de línea, sobre una conexión TCP persistente. Se eligió un protocolo propio y simple, solo con la biblioteca estándar, porque deja ver cada paso en las pruebas y en el informe; en producción se usaría HTTP o gRPC con TLS.

```
-> {"id":1,"tipo":"ENVIAR","clave":"FACTURA:SETP990000000:<cufe>","documento":{...}}
<- {"id":1,"ok":true,"aceptado":true,"codigo":"SIM-00","mensaje":"...","nodo":"nodo-
1","ms":41.2}
<- {"id":1,"ok":false,"error":"OCUPADO","nodo":"nodo-1"}      (cola de admisión llena)
-> {"id":2,"tipo":"ESTADO"}      (latido: hilos, cola, procesadas, saturadas,
duplicadas)
```

6.3 Hilos y procesos

Tabla 9
Hilos y procesos del sistema

Proceso	Hilo	Función
API (uvicorn)	Bucle de eventos y pool de hilos de FastAPI	Atiende las peticiones HTTP; cada emisión concurrente ocupa un hilo mientras espera al nodo.
API	`bus-eventos`	Entrega los eventos a los observadores.

Proceso	Hilo	Función
API	`vigilante-nodos`	Latidos cada 2 s: actualiza el circuito y la carga de cada nodo.
Nodo	`nodo-N-servidor`	Acepta conexiones nuevas.
Nodo	Un hilo por conexión	Lee solicitudes y las entrega al pool; espera el resultado.
Nodo	Pool `nodo-N-hilo` (4 por defecto)	Ejecuta el trabajo: valida y «transmite».

6.4 Cómo se simula el entorno distribuido

- **Nodos como procesos reales.** `ClusterLocal` lanza cada nodo con `python -m app.distribuido.nodo`; cada uno elige su puerto y lo informa por su salida estándar.
- **Red real.** La comunicación usa sockets TCP sobre el bucle local; cada nodo ve conexiones, desconexiones y lectura parcial como en una red real.
- **Latencia del proveedor.** Cada documento espera `latencia_ms` (40 ms, con 25 % de variación) para representar la firma y la transmisión.
- **Fallos inyectados.** Un nodo puede fallar con probabilidad configurable (`prob_error`), apagarse en caliente (`detener()`) o morir de forma abrupta con SIGKILL, que es lo que usan las mediciones de robustez.
- **Nodos en hilos para las pruebas.** Las pruebas arrancan nodos dentro de hilos del proceso de prueba, pero igualmente por TCP, para que sean rápidas y deterministas.

6.5 Integración con el sistema existente

La integración se hizo con cambios mínimos y verificables:

- **Configuración.** Variables `NODOS_PROVEEDOR`, `PROXY_TIMEOUT_SEG`, `PROXY_MAX_INTENTOS`, `CIRCUITO_FALLOS` y `CIRCUITO_ENFRIAMIENTO_SEG`. Vacía la primera, el comportamiento es el de la Unidad 1.
- **Composición.** `container.py` crea el clúster si hay nodos y le pasa su proxy al servicio como proveedor; el resto del sistema no cambia.
- **Ciclo de vida.** El arranque de la API inicia el bus y los latidos y el apagado los detiene.
- **Observabilidad.** `GET /api/electronica/nodos` (solo administrador) y el campo `nodos_facturacion` de `/health`.
- **Ajuste del servicio.** `FacturacionElectronicaService` separó la emisión en *reservar* (con candado) y *transmitir* (sin candado); ver sección 7.6.

6.6 Cómo ejecutarlo

```
cd backend
python -m app.distribuido.cluster --nodos 3     # imprime
NODOS_PROVEEDOR=127.0.0.1:P1,127.0.0.1:P2,127.0.0.1:P3
NODOS_PROVEEDOR=127.0.0.1:P1,127.0.0.1:P2,127.0.0.1:P3 uvicorn app.main:crear_app --
factory

pytest                                          # todas las pruebas
python benchmarks/benchmark_distribuido.py      # experimentos e1 a e4
python benchmarks/graficas.py                   # gráficas
```

6.7 Pruebas automáticas

El repositorio pasa de 155 a 181 pruebas. Las 26 nuevas ejercitan el camino de red real; se agrupan así:

Tabla 10
Pruebas del módulo distribuido

Grupo	Qué verifican	Pruebas
Protocolo	El documento viaja completo y la clave de idempotencia se conserva.	1
Nodo	Ping y estado, solicitudes inválidas, uso exacto del pool de hilos, contrapresión (OCUPADO) e idempotencia (secuencial y simultánea).	6
Proxy y mediador	Cumple la interfaz, rechazo del proveedor sin reintento, reparto entre 3 nodos, caída de un nodo, cierre del circuito al volver, ningún nodo, nodo lento, nodo saturado y error del nodo.	9
Observadores	Entrega, aislamiento de observadores rotos, descarte con cola llena y percentiles.	3
Integración con la API	Emisión por nodos, estado del clúster y permisos, API sin nodos y reintento al volver, emisión simultánea de una misma factura, ausencia de fuga de hilos.	5
Ajuste del servicio	Tres facturas distintas se emiten en paralelo sin repetir consecutivos; una excepción del proveedor deja el documento en RECHAZADO y se puede reintentar.	2

7. Evaluación del desempeño y ajustes

7.1 Metodología

Las mediciones se hicieron el 2026-11-11 en una máquina Linux de 2 núcleos y 8 GB de memoria con Python 3.13.18. Cada nodo es un proceso independiente; el generador de carga son hilos dentro del proceso de la API, que llaman al proxy como lo haría el servicio. Cada documento espera 40 ms (±25 %) en el nodo para representar la firma y la transmisión. Antes de medir se hace un calentamiento que abre las conexiones del pool, y los experimentos e1 a e3 se repiten 3 veces. Las latencias se miden en el cliente e incluyen los reintentos.

Tabla 11
Experimentos realizados

Id	Pregunta	Configuración	Carga
e1	¿Escala al agregar nodos?	1, 2 y 4 nodos con 4 hilos y cola de 64	600 documentos, 64 clientes
e2	¿Escala al agregar hilos?	1 nodo con 1, 2, 4, 8 y 16 hilos	400 documentos, 64 clientes
e3	¿Qué pasa al saturar y cómo influye la cola?	2 nodos de 2 hilos; cola de 1 frente a cola de 32	500 documentos, 64 clientes
e4	¿Tolera la caída de un nodo?	3 nodos; SIGKILL a nodo-2 a los 5 s y regreso a los 11 s	18 s continuos, 12 clientes
e5	¿El servicio completo aprovecha los nodos?	Servicio real, 1 y 4 nodos, código antes y después del ajuste	95 facturas, 16 clientes

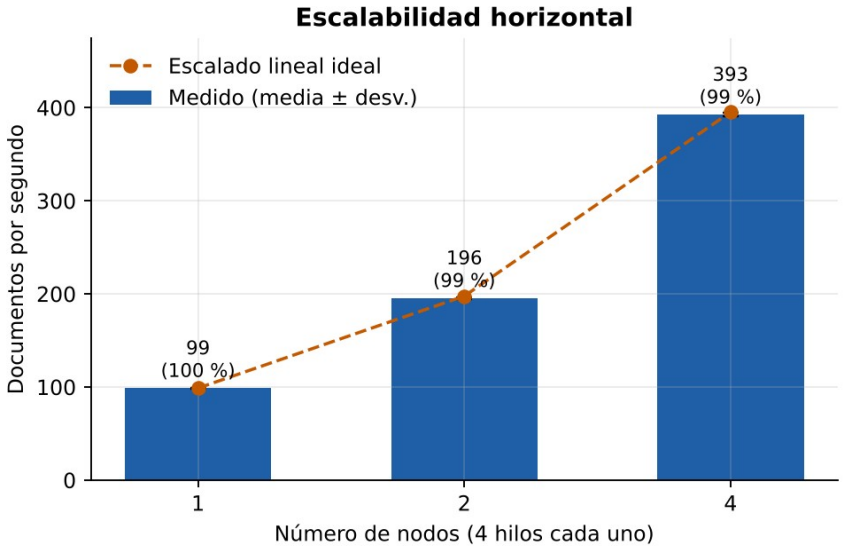
7.2 Escalabilidad horizontal (e1)

Tabla 12
Escalabilidad horizontal: nodos de 4 hilos cada uno

Nodos	Documentos/s (media ± desv.)	Aceleración	Eficiencia	p50 (ms)	p95 (ms)	p99 (ms)	Éxito
1	98,8 ± 0,3	1,00×	100,0 %	643	666	673	100 %
2	195,5 ± 1,0	1,98×	99,0 %	323	343	349	100 %
4	392,6 ± 2,0	3,97×	99,2 %	159	174	180	100 %

Nota. Tres repeticiones por configuración. Eficiencia = aceleración dividida por el número de nodos.

Figura 4
Documentos por segundo según el número de nodos



Nota. Barras: medición; línea punteada: escalado lineal ideal. Entre paréntesis, la eficiencia.

El sistema escala de forma casi lineal: con 4 nodos se alcanzan 392,6 documentos por segundo, 3,97 veces lo de un nodo (99,2 % de eficiencia), por encima de la meta del 90 %. La predicción de la ley de Little era de unos 100 documentos por segundo por nodo (4 hilos entre 40 ms) y se midieron 98,8. La latencia p95 baja de 666 a 174 ms porque los 64 clientes esperan menos en la cola: en un sistema cerrado, la latencia es aproximadamente el número de clientes entre el rendimiento (64 / 98,8 ≈ 648 ms).

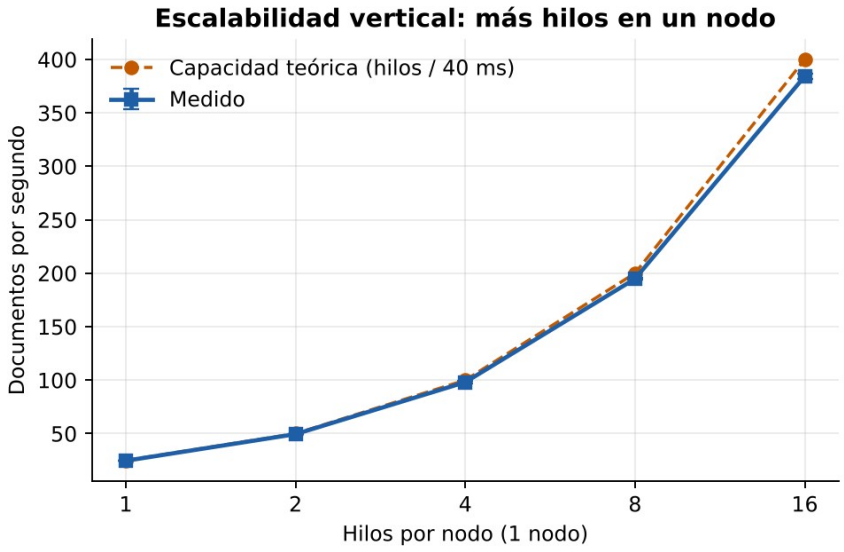
Alcance de la conclusión. La parte secuencial del trabajo (elegir nodo y serializar) es mínima frente a los 40 ms de espera, así que la ley de Amdahl (Amdahl, 1967) apenas limita el escalado. Con un trabajo que consumiera CPU en la API, esa fracción serial pesaría mucho más, como muestra la sección 7.6.

7.3 Escalabilidad vertical: hilos por nodo (e2)

Tabla 13
Escalabilidad vertical: un nodo con distinto número de hilos

Hilos	Documentos/s medido	Teórico (hilos / 40 ms)	Cumplimiento	p50 (ms)	p95 (ms)
1	24,6	25,0	98,4 %	2583	2668
2	49,5	50,0	99,0 %	1286	1332
4	98,1	100,0	98,1 %	646	672
8	194,8	200,0	97,4 %	322	338
16	384,3	400,0	96,1 %	160	172

Figura 5
Documentos por segundo según los hilos de un nodo



Nota. Línea punteada: capacidad teórica h / W con W = 40 ms.

El rendimiento de un solo nodo sigue la ley de Little casi exactamente (entre 98 % y 96 % de lo teórico) hasta 16 hilos. Es lo esperado para un trabajo que casi no usa CPU: los hilos pasan su tiempo esperando y el GIL no estorba. Para un trabajo que sí calculara (por ejemplo, firmar con XAdES), más hilos dejarían de ayudar al saturar un núcleo y habría que agregar nodos, que son procesos y no comparten el GIL. Esa es la razón de combinar ambos niveles de concurrencia.

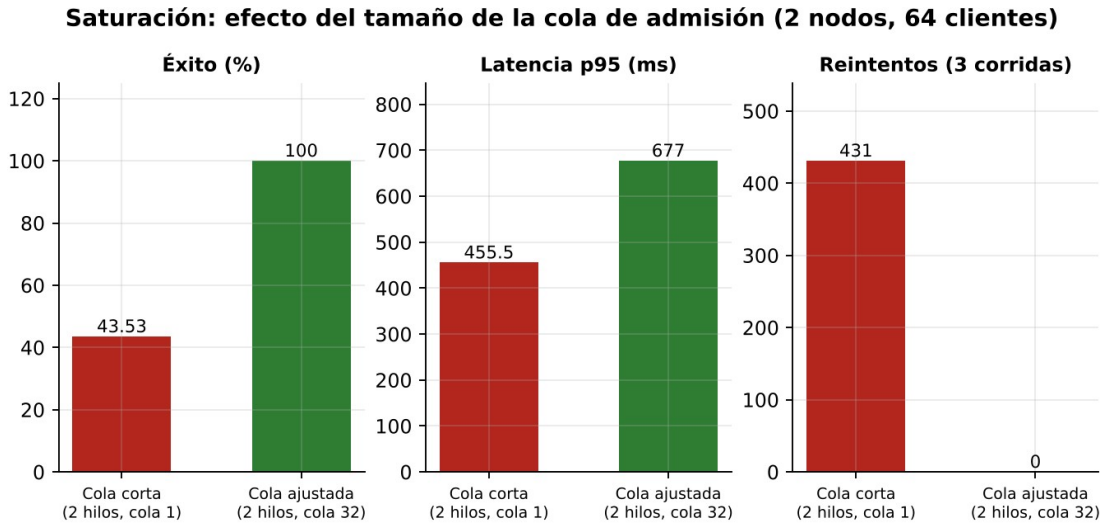
7.4 Saturación y contrapresión (e3)

Tabla 14
Efecto del tamaño de la cola de admisión (2 nodos de 2 hilos, 64 clientes)

Configuración	Éxito	Docs/s totales	Docs/s aceptados	p95 (ms)	Reintentos	Intentos fallidos
Cola corta (cola = 1)	43,5 %	216,7	94,3	456	431	946 OCUPADO, 327 sin nodos
Cola ajustada (cola = 32)	100,0 %	97,9	97,9	677	0	ninguno

Nota. Los reintentos y los intentos fallidos suman las tres repeticiones. «Docs/s aceptados» = documentos por segundo × tasa de éxito.

Figura 6
Éxito, latencia p95 y reintentos según la cola de admisión



La capacidad de los dos nodos es de unos 100 documentos por segundo y la carga de 64 clientes la supera de lejos. Con la cola corta, los nodos se protegen respondiendo OCUPADO: el proxy reintenta en el otro nodo, pero tras 4 intentos fallidos entrega un rechazo NODOS-01 y solo el 43,5 % de los documentos se acepta a la primera. Los documentos aceptados por segundo (94,3) son, sin embargo, casi los mismos que con la cola ajustada (97,9): la capacidad real no cambia; cambia qué se hace con el exceso. La latencia p95 de la cola corta (456 ms) parece mejor, pero es engañosa porque incluye los envíos que fallan rápido.

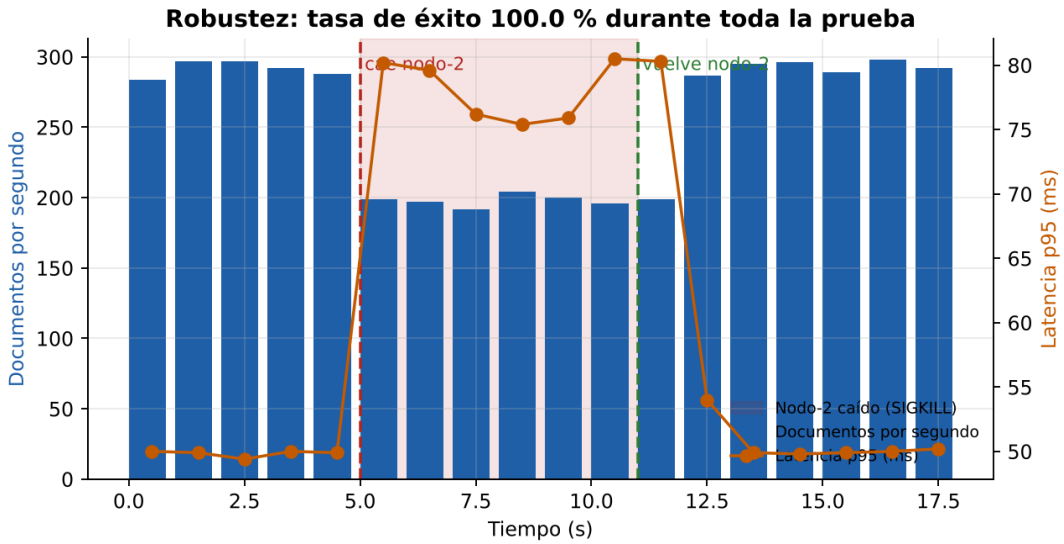
Con la cola de 32, el exceso espera en la cola del nodo: el éxito sube al 100 % sin un solo reintento y la latencia sube a unos 677 ms (≈ 64 clientes / 98 documentos por segundo). Es un compromiso de diseño legítimo: la cola corta protege la latencia y delega el exceso al «Reintentar» de la aplicación (los documentos quedan RECHAZADOS, no se pierden); la cola larga absorbe ráfagas a costa de latencia. El valor por defecto (cola de 16) y la recomendación de dimensionarla según la ráfaga esperada salen de este experimento.

7.5 Robustez ante la caída de un nodo (e4)

Tabla 15
Caída y recuperación de un nodo (3 nodos de 4 hilos, 12 clientes)

Indicador	Valor
Documentos enviados en 18 s	4614
Documentos aceptados (tasa de éxito)	4614 (100,0 %)
Reintentos totales (todos por conexión rechazada)	7
Detección: caída del nodo → circuito ABIERTO	0,01 s
Recuperación: nodo de regreso → circuito CERRADO	0,96 s
Rendimiento antes de la caída	293,5 docs/s
Rendimiento con el nodo caído	197,8 docs/s (67 % del normal)
Rendimiento tras la recuperación	294,0 docs/s
Latencia p95 por segundo, antes → con la caída	50 → 76 ms
Latencia máxima de cualquier documento	187 ms

Figura 7
Rendimiento y latencia p95 por segundo durante la caída de nodo-2



Nota. La franja roja marca el intervalo en que nodo-2 estuvo caído (SIGKILL a los 5 s; reinicio a los 11 s).

Ningún documento se perdió ni quedó sin respuesta (100,0 % de éxito). El circuito del nodo caído se abrió 0,01 s después de la caída y solo hicieron falta 7 reintentos, porque tras abrirse el circuito el mediador deja de enviarle tráfico. Con un nodo de tres fuera, el rendimiento cae al 67 % del normal, proporcional a la capacidad perdida (2/3), y la latencia p95 sube de unos 50 a unos 76 ms: degradación proporcional y no un colapso. Mientras el nodo está caído, su circuito pasa a SEMIABIERTO una vez por segundo y vuelve a abrirse; cuesta una sola solicitud de prueba por segundo. Cuando el nodo regresa, el circuito se cierra 0,96 s después (aproximadamente el enfriamiento de 1 s configurado para esta prueba) y el rendimiento vuelve al nivel anterior sin reiniciar la API.

Este resultado cumple la meta del 100 % de éxito con la caída de 1 de 3 nodos y la recuperación automática. Conviene recordar el límite del experimento: se probó la caída por parada (SIGKILL), no la partición de red ni un nodo que responde mal pero no cae.

7.6 Servicio completo y el ajuste del candado (e5)

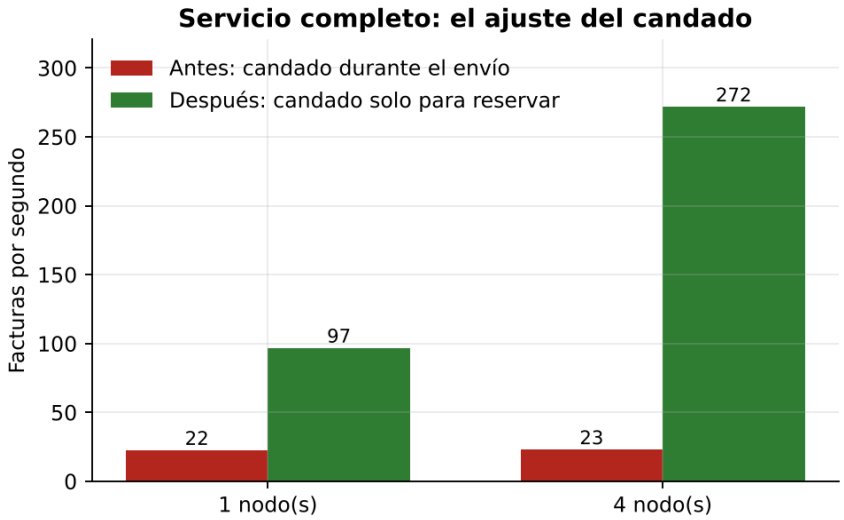
Los experimentos anteriores miden el proxy y los nodos. El experimento e5 mide el sistema completo: 16 clientes llaman a `FacturacionElectronicaService.emitir` sobre facturas distintas, como lo harían varios cajeros a la vez. La primera medición, con el código tal como estaba, reveló un problema que las anteriores no mostraban.

Tabla 16
Emisiones simultáneas por el servicio completo: antes y después del ajuste

Nodos	Antes (facturas/s)	Antes p50 (ms)	Después (facturas/s)	Después p50 (ms)	Mejora
1	22,4	697	96,7	159	4,3×
4	22,9	697	271,7	51	11,9×

Nota. En todas las mediciones el éxito fue del 100 % y los consecutivos quedaron sin repetir.

Figura 8
Facturas por segundo antes y después de separar la reserva del envío



Cuello de botella. Con 1 y con 4 nodos el servicio entregaba lo mismo (22,4 y 22,9 facturas por segundo): agregar nodos no servía de nada. La causa estaba en el servicio: `emitir` tomaba un candado para numerar el documento y lo mantenía mientras esperaba la respuesta del proveedor, así que todas las emisiones se hacían una tras otra (cada una tardaba cerca de 44 ms, y el p50 de 697 ms es la espera en fila de 16 clientes). Era un cuello de botella de diseño que ninguna cantidad de nodos podía arreglar.

Ajuste. Se separó la emisión en dos pasos. Reservar (con candado): validar, numerar, calcular el CUFE y el XML y guardar el documento en estado GENERADO. Transmitir (sin candado): enviar al proveedor y guardar su respuesta. El documento reservado impide que otra emisión de la misma factura se cuele y el consecutivo ya no puede repetirse. Si el proveedor lanza una excepción, el documento pasa a RECHAZADO (ERR-PROVEEDOR) para no quedar «en curso» eternamente. Las 155 pruebas previas y las nuevas siguen pasando.

Resultado. Con 1 nodo se pasa de 22,4 a 96,7 facturas por segundo (4,3×, casi la capacidad de 100 que predice la ley de Little para 4 hilos) y con 4 nodos de 22,9 a 271,7 (11,9×); la latencia mediana cae de 697 a 51 ms.

Nuevo límite. Con 4 nodos se llegan a 271,7 facturas por segundo, por debajo de los 393 que admiten los nodos. Para ubicar el siguiente cuello se repitió la medición con latencia simulada cero, es decir, sin esperar a ningún nodo: la API sola procesó entre 317 y 365 facturas por segundo. El techo es ahora el trabajo de CPU del propio proceso de la API (generar el XML, calcular el CUFE SHA-384 y copiar objetos en los repositorios en memoria), que además comparte el GIL con los hilos de las peticiones. Es una medición de orden de magnitud (95 facturas por corrida), no un valor preciso.

7.7 Cuellos de botella y ajustes

Tabla 17
Resumen de cuellos de botella, evidencia y ajustes

Cuello de botella	Evidencia	Ajuste	Efecto
Candado del servicio retenido durante el envío	e5: 22,4 y 22,9 facturas/s con 1 y 4 nodos	Separar reservar (con candado) de transmitir (sin candado)	11,9× con 4 nodos; consecutivos sin repetir
Cola de admisión demasiado corta frente a ráfagas	e3: éxito 43,5 % con cola de 1	Cola dimensionada según la ráfaga (32 en la prueba; 16 por defecto)	Éxito 100 %, a cambio de latencia p95 de 677 ms
Reintentos sobre un nodo caído	e4: 7 reintentos en 4614 documentos	Circuit breaker: el nodo caído deja de recibir tráfico	Detección en 0,01 s; éxito 100 %
CPU y GIL del proceso de la API (nuevo techo)	e5 sin latencia: 317 a 365 facturas/s	Pendiente: más procesos de API con base de datos real, o mover la generación del XML a workers	No aplicado (trabajo futuro)

7.8 Resultado de las pruebas automáticas

Al cierre del trabajo, la suite completa pasa: 181 pruebas (155 de la Unidad 1 y 26 del módulo distribuido) en 23 segundos. Las pruebas del módulo se ejecutan por TCP real, incluyen la caída y recuperación de nodos, y verifican propiedades que las mediciones solo muestran de forma indirecta (idempotencia, contrapresión, ausencia de fuga de hilos).

7.9 Límites y amenazas a la validez

- **Proveedor simulado.** La latencia es una espera fija con variación; el proveedor real tendría colas, errores y latencias de cola larga. Por eso se reportan p95 y p99, que dominan la experiencia en sistemas distribuidos (Dean y Barroso, 2013).
- **Red local.** Los nodos hablan por el bucle local: no hay pérdida de paquetes, retardo real ni particiones. Se probó la caída de un proceso, no de la red.
- **Máquina pequeña y compartida.** Los 2 núcleos los comparten generador de carga, API y nodos. Como los nodos casi no usan CPU (esperan), el efecto es pequeño, pero no nulo.
- **Carga de lazo cerrado.** Cada cliente envía el siguiente documento al recibir la respuesta; una carga de llegadas independientes (lazo abierto) podría mostrar colas distintas.
- **Trabajo limitado por espera.** Las conclusiones de escalado de hilos valen para ese tipo de trabajo; para uno limitado por CPU habría que repetir e2.
- **Repeticiones.** e1 a e3 se repitieron 3 veces; e4 y e5 se midieron una vez por configuración, por lo que sus cifras son indicativas.
- **Datos en memoria.** El servicio y los repositorios no se reparten: hay una sola API. La escalabilidad medida es la de la facturación electrónica, no la de todo el sistema.

8. Conclusiones

El mini proyecto cumplió su objetivo: se diseñó, modeló, implementó y evaluó una arquitectura distribuida real para una parte del consultorio «Salud Integral», sin romper el sistema existente. Las conclusiones principales son:

- **La arquitectura escala.** Con nodos de 4 hilos, el rendimiento crece de 98,8 a 392,6 documentos por segundo de 1 a 4 nodos (99,2 % de eficiencia) y, dentro de un nodo, hasta 16 hilos siguen la ley de Little.
- **Tolera fallos.** Con la caída de un nodo en plena carga no se perdió ningún documento (100,0 % de éxito), la detección tardó 0,01 s y el sistema se recuperó solo cuando el nodo volvió.
- **Los patrones resolvieron problemas concretos.** Proxy y Adapter permitieron distribuir sin tocar el servicio; Mediator concentró el balanceo y la salud de los nodos; Observer midió sin frenar; Circuit Breaker e Idempotent Receiver hicieron seguros los reintentos.
- **Medir encontró lo que el diseño no mostraba.** El mayor hallazgo no estaba en los nodos sino en el candado del servicio, que anulaba todo el escalado. Corregirlo multiplicó el rendimiento por 11,9 con 4 nodos.

- **La contrapresión es una decisión de producto.** Acotar la cola protege la latencia pero convierte el exceso en rechazos reintentables; ampliarla acepta todo a costa de espera. La elección depende de qué es peor para el consultorio.

8.1 Reflexión sobre las decisiones y los desafíos

- **Elegir la facturación electrónica** fue acertado: es la única dependencia remota del sistema, y por eso la distribución tiene sentido real y no es un ejercicio artificial.
- **Pensar en fallos parciales.** La mayor dificultad conceptual es que una llamada remota puede haberse ejecutado aunque no haya respuesta. Resolverlo exigió idempotencia y claves, no solo reintentos.
- **Probar la concurrencia.** Las pruebas con nodos en hilos por TCP real fueron la forma de hacerlas rápidas y repetibles; las mediciones con procesos reales aportaron lo que las pruebas no pueden (SIGKILL y tiempos de recuperación).
- **Honestidad en la evaluación.** Un resultado como «el escalado es del 99 %» solo es creíble junto a sus límites: proveedor simulado, red local, trabajo limitado por espera.

8.2 Trabajo futuro

- Reemplazar el proveedor simulado por un adaptador de un proveedor tecnológico autorizado por la DIAN, que ya tiene su puerto (`IProveedorTecnologico`).
- Cifrar y autenticar la comunicación entre la API y los nodos (TLS mutuo) y registrar los nodos de forma dinámica.
- Persistir los datos en una base de datos real y escalar la API en varias réplicas, que es el siguiente techo medido; con ella, usar un *outbox* transaccional para no perder envíos si la API se cae entre reservar y transmitir.
- Reducir la latencia de cola larga con solicitudes redundantes al segundo nodo (Dean y Barroso, 2013) y medir con carga de lazo abierto.
- Empaquetar cada nodo en un contenedor y desplegar varios en la nube.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings, 30*, 483–485. https://doi.org/10.1145/1465482.1465560

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Dean, J., & Barroso, L. A. (2013). The tail at scale. *Communications of the ACM, 56*(2), 74–80. https://doi.org/10.1145/2408776.2408794

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Little, J. D. C. (1961). A proof for the queuing formula: L = λW. *Operations Research, 9*(3), 383–387. https://doi.org/10.1287/opre.9.3.383

Microsoft. (s. f.). *Cloud design patterns*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

Python Software Foundation. (s. f.-a). *concurrent.futures — Launching parallel tasks*. Python 3 documentation. https://docs.python.org/3/library/concurrent.futures.html

Python Software Foundation. (s. f.-b). *socketserver — A framework for network servers*. Python 3 documentation. https://docs.python.org/3/library/socketserver.html

Tanenbaum, A. S., & van Steen, M. (2017). *Distributed systems* (3.ª ed.). Maarten van Steen.

Anexo A. Trazabilidad con los criterios de evaluación

Tabla 18
Criterios de evaluación y evidencia

Criterio	Evidencia	Sección
Diseño de la arquitectura distribuida	Diagramas de componentes, secuencia y despliegue; requisitos RFD y RNFD	3 y 4
Implementación del prototipo	Paquete `app/distribuido`, integración con la API, pruebas automáticas	6
Uso de patrones de diseño	Proxy, Mediator, Observer, Circuit Breaker, Retry, Idempotent Receiver y pool de hilos	5
Evaluación de escalabilidad y robustez	Experimentos e1 a e5, cuellos de botella y ajustes	7
Presentación en video	Video no listado en YouTube de 12 a 15 minutos; guion en `docs/video/guion_video.md`	Anexo B
Trabajo en equipo	Roles, estrategia y cronograma	3.4 y 3.5

Anexo B. Enlaces de la entrega

Tabla 19
Enlaces

Producto	Enlace
Repositorio del código fuente	https://github.com/JNACHO2001/consultorio-medico
Aplicación en línea (modo local)	https://consultorio-medico-pty2.onrender.com
Video en YouTube (no listado)	[Pegar aquí el enlace del video una vez subido]